Distinction Graph Machines: Refine-or-Repair Runtime Learning

Anonymous

Abstract

We introduce Distinction Graph Machines, a runtime-learning model whose state is a growing graph of prototypes, local memories, and pairwise distinctions. The guiding rule is refine when the current concepts are too coarse, and repair when a local concept is adequate but its state is wrong. The theory separates these operations. Minimal refinement is the smallest partition or sigma-algebra update that makes new evidence measurable, and contradictions inside an atom cannot be solved by any predictor measurable with respect to the current information. The value of a refinement is exact: under log loss it is conditional mutual information, and under squared loss it is conditional variance reduction. The concrete data structure realizes refinement by creating prototypes and distinction edges, while local memories are repaired by sufficient-statistic, exemplar, ridge, or Projection Memory updates. We give prediction and observation algorithms, a data-structure complexity theorem, and a language-modeling instance in which hidden-state prototypes store next-token memories and produce runtime logit corrections without inference-time backpropagation. We further define a DGM Transformer layer with differentiable sparse reads and detached local writes, allowing slow parameters to be trained by backpropagation while the adaptive state is updated by the same forward-only rule used at inference. Projection Memory and CPM appear as local repair mechanisms after the relevant distinction structure has been fixed.

Contents

1 Introduction **1**

2 Preliminaries **1**
2.1 Notation . . . 1
2.2 Partitions and measurable predictors . . . 2
2.3 Bayes risk under information . . . 2
2.4 Projection memory notation . . . 2

3 Minimal Information Refinement **2**
3.1 The finite refinement rule . . . 3
3.2 The sigma-algebra form . . . 3
3.3 Contradiction inside an atom . . . 3

4 The Value of a Distinction **3**
4.1 Risk monotonicity . . . 4
4.2 Log loss . . . 4
4.3 Squared loss . . . 4
4.4 Finite empirical scores . . . 4

5 Distinction Graph Machines **5**
5.1 State and prediction . . . 5
5.2 Refine or repair on a graph . . . 6
5.3 Why graph memory is different from a tree . . . 7
5.4 Local memories . . . 7
5.5 Language modeling instance . . . 7
5.6 Complexity . . . 8

6 DGM Layers for Large-Model Training **9**
6.1 A trainable DGM layer . . . 9
6.2 Detached write path . . . 9
6.3 Why this can scale . . . 10
6.4 Distributed training . . . 10
6.5 Closed-form adapters . . . 11

7 From Distinctions to Projection Memory **11**
7.1 One-hot partition memory . . . 11
7.2 CPM conflict as missing distinction . . . 12
7.3 The hierarchy . . . 12

8 Technical Overview **12**

9 Computational Checks **15**

10 Conclusion **17**

A Appendix Roadmap **20**

B Limitations **20**

C Related Work **20**

D Minimal Refinement Proofs **21**

E Value of Distinction Proofs **23**

F DGM Algorithm and Projection-Bridge Proofs **25**

1 Introduction

Runtime learning has two different failure modes. Sometimes the model is looking at the right kind of state, but the state is numerically wrong; then the right operation is repair. Sometimes the model is using an information structure that identifies situations whose consequences differ; then no local repair can solve the error. The right operation is refinement.

This paper builds a model around that distinction. A Distinction Graph Machine keeps a growing set of prototypes, local memories, and distinction edges. The prototypes define what the learner currently treats as concepts. The edges record which concepts had to be separated. The local memories store what the learner currently believes inside each concept. On an input x, an encoder produces $h = \phi_\theta(x)$, the machine retrieves nearby prototypes, and their local memories predict. When the consequence y is observed, the machine decides whether to create a new prototype and distinction edges, or repair the memory of an existing prototype.

The guiding rule is

if the atom is wrong, refine; if the local state is wrong, repair.

Refinement creates a new concept when existing concepts are too coarse. Repair updates a local model inside the existing graph. For a simple classifier, repair can be an exemplar or sufficient-statistic update. For a richer local memory, repair can be Projection Memory or CPM.

The theory explains why the two operations should not be collapsed. If two states lie in the same atom but require different outputs, no measurable predictor over the current partition can satisfy both. This is not an optimization failure; it is a missing distinction. Conversely, if the current concept is adequate and only its local state is stale, creating a new concept is unnecessary. The model should update the local memory.

We formalize both sides. Minimal information refinement is the smallest partition or sigma-algebra update that makes new evidence observable. Its value is exact under standard losses: conditional mutual information for log loss and conditional variance reduction for squared loss. The DGM data structure adds the engineering layer missing from the pure refinement principle: prototypes, local memories, distinction edges, repair rules, language-modeling integration, and train/test APIs. We also describe a DGM Transformer layer whose read path is differentiable and whose write path is detached, so that large models can learn to use DGM memory during training without backpropagating through the entire memory history.

Organization. Section 2 fixes notation and information structures. Section 3 proves the refinement principle and the contradiction theorem. Section 4 quantifies the value of distinctions. Section 5 defines DGM as a concrete model and algorithm, including a language-modeling instance. Section 6 gives a scalable DGM layer for large-model training. Section 7 relates local repair to Projection Learning and CPM. Section 8 gives proof sketches. Section 9 gives executable checks. Section 10 concludes.

2 Preliminaries

2.1 Notation

All probability spaces are denoted by $(\Omega, \mathcal{A}, \mathbb{P})$ unless the finite case is stated explicitly. Random variables are functions on Ω. For vectors $x \in \mathbb{R}^d$, lengths use the ℓ_2 norm $\|x\|_2$. For matrices,

$\|M\|_F$ is the Frobenius norm. Inner products are written $\langle x, y\rangle$. For a matrix K, the Moore-Penrose pseudoinverse is $K^\dagger$, and $KK^\dagger$ is the orthogonal projector onto the span of the columns of K.

For sigma-algebras $\mathcal{F}, \mathcal{G} \subseteq \mathcal{A}$, the inclusion $\mathcal{F} \subseteq \mathcal{G}$ means that $\mathcal{G}$ is at least as informative as $\mathcal{F}$. The join $\mathcal{F} \vee \mathcal{G}$ is the smallest sigma-algebra containing both. For a random variable Z, $\sigma(Z)$ is the sigma-algebra generated by Z; for an event E, $\sigma(E) = \{\varnothing, E, E^c, \Omega\}$.

2.2 Partitions and measurable predictors

In the finite setting, an information structure is a partition

$$\mathcal{P} = \{B_1, \ldots, B_n\}$$

of Ω into nonempty disjoint atoms. A partition $\mathcal{Q}$ refines $\mathcal{P}$ if every atom of $\mathcal{Q}$ is contained in an atom of $\mathcal{P}$. An event $E \subseteq \Omega$ is $\mathcal{P}$-measurable if it is a union of atoms of $\mathcal{P}$.

A prediction or action $f : \Omega \to A$ is $\mathcal{P}$-measurable if it is constant on each atom:

$$\omega, \omega' \in B \in \mathcal{P} \quad \Longrightarrow \quad f(\omega) = f(\omega').$$

Equivalently, f can depend only on which atom contains the current world state. In the sigma-algebra setting, admissible predictors are $\mathcal{F}$-measurable functions.

2.3 Bayes risk under information

Let $Y : \Omega \to \mathcal{Y}$ be a target random variable, let A be an action space, and let $L : \mathcal{Y} \times A \to \mathbb{R}$ be a loss for which the expectations below are well-defined. The Bayes risk available under information $\mathcal{F}$ is

$$R_L(\mathcal{F}) = \inf\{\mathbb{E}[L(Y, a)] : a : \Omega \to A,\ a \text{ is } \mathcal{F}\text{-measurable}\}. \tag{1}$$

Thus $R_L(\mathcal{F})$ is the best population risk achievable by a predictor that can use only distinctions in $\mathcal{F}$.

2.4 Projection memory notation

For the bridge to Projection Learning, we use a finite partition $\mathcal{P} = \{B_1, \ldots, B_n\}$ and its one-hot feature map $\phi_{\mathcal{P}} : \Omega \to \mathbb{R}^n$:

$$\phi_{\mathcal{P}}(\omega) = e_i \qquad \text{if} \qquad \omega \in B_i.$$

A matrix $W \in \mathbb{R}^{m \times n}$ assigns a value vector to each atom through $W\phi_{\mathcal{P}}(\omega)$. For old key matrices K, a new key k has CPM innovation

$$z = (I - KK^\dagger)k. \tag{2}$$

3 Minimal Information Refinement

The primitive operation in this paper is refinement. The learner has an information structure and receives evidence that a new event or variable must become observable. The correct update should add that observability and nothing else. This section states the finite theorem first because it is the clearest form of the idea: split only the atoms cut by the evidence.

3.1 The finite refinement rule

Let $\mathcal{P}$ be a finite partition and let $E \subseteq \Omega$ be a new binary distinction. Define

$$\begin{aligned}\mathcal{P}_E = \{B \cap E : B \in \mathcal{P}, B \cap E \neq \varnothing\} \\ \cup \{B \cap E^c : B \in \mathcal{P}, B \cap E^c \neq \varnothing\}.\end{aligned} \tag{3}$$

This notation simply keeps the nonempty pieces obtained by cutting each old atom by E.

Theorem 3.1 (Minimal finite refinement, informal version of Theorem D.1)**.** *Let $\mathcal{P}$ be a finite partition of Ω and let $E \subseteq \Omega$. The partition $\mathcal{P}_E$ is the unique minimal partition that refines $\mathcal{P}$ and makes E measurable. More precisely, if $\mathcal{Q}$ refines $\mathcal{P}$ and E is $\mathcal{Q}$-measurable, then $\mathcal{Q}$ refines $\mathcal{P}_E$.*

The theorem gives a canonical update. We do not search over all refined representations; the evidence itself determines the least one. An atom untouched by E is left unchanged. An atom crossed by E is split into the two pieces that the evidence can now distinguish.

3.2 The sigma-algebra form

For general spaces, the same operation is the join. If the current information is $\mathcal{F}_t$ and the evidence is a variable Z_t, the minimal updated information is

$$\mathcal{F}_{t+1} = \mathcal{F}_t \vee \sigma(Z_t). \tag{4}$$

Theorem 3.2 (Minimal sigma-algebra refinement, informal version of Theorem D.2)**.** *Let $\mathcal{F} \subseteq \mathcal{A}$ be a sigma-algebra and let Z be a random variable. The join $\mathcal{F} \vee \sigma(Z)$ is the smallest sigma-algebra $\mathcal{G}$ such that $\mathcal{F} \subseteq \mathcal{G}$ and Z is $\mathcal{G}$-measurable.*

Thus the same object governs finite decision-tree splits and measurable-space refinement. A distinction is not an auxiliary feature added after learning; it is a change in the sigma-algebra of what the learner can condition on.

3.3 Contradiction inside an atom

Minimal refinement is necessary when the current information identifies states that the target separates. This is the simplest obstruction to pure optimization.

Theorem 3.3 (Contradiction requires distinction, informal version of Theorem D.3)**.** *Let $\mathcal{P}$ be a partition of Ω and let $y : \Omega \to A$ be a required output. Suppose ω, ω' lie in the same atom of $\mathcal{P}$ but $y(\omega) \neq y(\omega')$. Then no $\mathcal{P}$-measurable predictor $f : \Omega \to A$ satisfies both $f(\omega) = y(\omega)$ and $f(\omega') = y(\omega')$.*

The conclusion is structural, not algorithmic. A larger step size, a different optimizer, or more iterations cannot produce two different outputs on states that the representation forces to be the same. The missing operation is not descent. It is refinement.

4 The Value of a Distinction

Refinement increases what the learner can condition on. This does not mean that every distinction is useful for every task. The value of a distinction is measured by the decrease in optimal risk after the information structure is refined.

4.1 Risk monotonicity

The first fact is purely order-theoretic. If $\mathcal{G}$ contains all events in $\mathcal{F}$, then every $\mathcal{F}$-measurable predictor is also $\mathcal{G}$-measurable. The decision class has only grown.

Theorem 4.1 (Bayes-risk monotonicity, informal version of Theorem E.1)**.** *If $\mathcal{F} \subseteq \mathcal{G}$, then for every loss L,*

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

This is a population statement. It says that information cannot hurt the best possible decision rule. Statistical estimation, finite samples, and model selection can still overfit; those are algorithmic questions, not failures of the information order.

4.2 Log loss

For a finite target variable Y, log loss asks the learner to output a conditional distribution. The optimal prediction under $\mathcal{F}$ is the conditional law of Y given $\mathcal{F}$, and the optimal risk is conditional entropy.

Theorem 4.2 (Log-loss value, informal version of Theorem E.2)**.** *Let Y be finite and let the loss be $-\log q(Y)$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}).$$

Thus, for classification and language modeling, a refinement is useful exactly to the extent that it carries conditional mutual information about the consequence beyond what the learner already knows.

4.3 Squared loss

For $Y \in L^2(\Omega; \mathbb{R}^d)$ and squared loss, the optimal predictor under $\mathcal{F}$ is $\mathbb{E}[Y \mid \mathcal{F}]$. Refinement helps precisely when it changes the conditional mean.

Theorem 4.3 (Squared-loss value, informal version of Theorem E.3)**.** *Let $Y \in L^2(\Omega; \mathbb{R}^d)$ and let $L(Y, a) = \|Y - a\|_2^2$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_2(\mathcal{F}) - R_2(\mathcal{G}) = \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].$$

The identity is the Hilbert-space analogue of the log-loss result. A distinction has squared-loss value when the cells it creates have different conditional means.

4.4 Finite empirical scores

The population identities lead directly to split scores. Suppose an atom B contains samples (x_j, y_j) and a candidate distinction $g : B \to \{0, 1\}$ splits it into B_0 and B_1.

For classification, let $\hat{p}_B$, $\hat{p}_{B_0}$, and $\hat{p}_{B_1}$ be empirical label distributions. The empirical log-loss improvement is

$$\Delta_{\log}(B, g) = |B| H(\hat{p}_B) - |B_0| H(\hat{p}_{B_0}) - |B_1| H(\hat{p}_{B_1}).$$

Equivalently,

$$\Delta_{\log}(B, g) = |B| \widehat{I}(Y; g \mid B).$$

For regression, define

$$\hat{\mu}_B = \frac{1}{|B|} \sum_{j \in B} y_j, \qquad \mathrm{SSE}(B) = \sum_{j \in B} \|y_j - \hat{\mu}_B\|_2^2 .$$

The empirical squared-loss improvement is

$$\Delta_2(B, g) = \mathrm{SSE}(B) - \mathrm{SSE}(B_0) - \mathrm{SSE}(B_1).$$

It also has the between-cell form

$$\Delta_2(B, g) = |B_0| \, \|\hat{\mu}_{B_0} - \hat{\mu}_B\|_2^2 + |B_1| \, \|\hat{\mu}_{B_1} - \hat{\mu}_B\|_2^2 .$$

These scores implement the same principle as the population theory: split only when the proposed distinction has measurable consequence value.

5 Distinction Graph Machines

The tree model is a useful finite special case, but it is not the most natural runtime data structure. A tree makes every decision through one irreversible path. A more flexible object is a graph of concepts: local memories live at prototypes, and distinctions live on edges between prototypes that have been confused. This section defines that model.

5.1 State and prediction

Let $\phi_\theta : \mathcal{X} \to \mathbb{R}^d$ be an encoder. A Distinction Graph Machine has runtime state

$$\mathcal{S}_t = \left(\{c_j, M_j, n_j\}_{j=1}^{N_t}, G_t \right).$$

Here $c_j \in \mathbb{R}^d$ is a prototype, M_j is a local predictor or memory, n_j is a usage count, and G_t is a graph on the prototypes. An edge (i, j) stores a distinction separating the two concepts, typically

$$u_{ij} = \frac{c_i - c_j}{\|c_i - c_j\|_2}, \qquad b_{ij} = \frac{1}{2} u_{ij}^\top (c_i + c_j).$$

The edge distinction is

$$g_{ij}(h) = \mathbf{1}\{u_{ij}^\top h > b_{ij}\}.$$

Given $h = \phi_\theta(x)$, the machine computes distances $d_j(h)$ to prototypes and a sparse neighborhood

$$N_k(h) \subseteq \{1, \ldots, N_t\}.$$

It then forms routing weights

$$\alpha_j(h) = \frac{\exp(-d_j(h)/T)}{\sum_{r \in N_k(h)} \exp(-d_r(h)/T)} \qquad j \in N_k(h),$$

and predicts by local mixture

$$\hat{y} = \sum_{j \in N_k(h)} \alpha_j(h) M_j(h).$$

Hard nearest-prototype routing is the limit $T \to 0$.

Algorithm 1 PREDICT for a Distinction Graph Machine

Input: state $\mathcal{S}_t = (\{c_j, M_j, n_j\}_{j=1}^{N_t}, G_t)$, encoder ϕ_θ, example x
$h \leftarrow \phi_\theta(x)$
$N_k(h) \leftarrow$ the k nearest prototypes to h
compute weights $\alpha_j(h)$ for $j \in N_k(h)$
return $\sum_{j \in N_k(h)} \alpha_j(h) M_j(h)$

5.2 Refine or repair on a graph

After observing consequence y, DGM first asks whether an existing concept can absorb the evidence. Let $j^\star$ be the nearest or highest-weight prototype. Define a local inconsistency score

$$\rho_t = L(y, M_{j^\star}(h)).$$

If ρ_t is small, the information structure is treated as adequate and the machine repairs the local state:

$$M_{j^\star} \leftarrow \text{Repair}(M_{j^\star}, h, y), \qquad c_{j^\star} \leftarrow c_{j^\star} + \eta_c(h - c_{j^\star}), \qquad n_{j^\star} \leftarrow n_{j^\star} + 1.$$

If ρ_t is large and the neighborhood contains incompatible consequences, the machine refines by creating a new prototype

$$c_{N_t+1} = h, \qquad M_{N_t+1} = \text{InitMemory}(h, y),$$

and adds distinction edges from the new prototype to the confused neighbors.

Algorithm 2 OBSERVE for a Distinction Graph Machine

Input: state $\mathcal{S}_t$, labeled consequence (x, y)
$h \leftarrow \phi_\theta(x)$
if $N_t = 0$ **then**
    create prototype $c_1 = h$ with memory InitMemory(h, y)
    return updated state
end if
$j^\star \leftarrow \text{argmin}_j\, d_j(h)$
$\rho \leftarrow L(y, M_{j^\star}(h))$
if $\rho \le \tau_{\text{repair}}$ **then**
    repair $M_{j^\star}$ and move $c_{j^\star}$ toward h
else
    create a new prototype $c_{N_t+1} = h$
    initialize M_{N_t+1} from (h, y)
    for confused neighbor i of h **do**
        add edge $(i, N_t + 1)$ with distinction $(u_{i,N_t+1}, b_{i,N_t+1})$
    end for
end if
return updated state

Theorem 5.1 (Prototype distinction separates its endpoints, informal version of Theorem F.1)**.** *If $c_i \neq c_j$ and u_{ij}, b_{ij} are defined as above, then*

$$u_{ij}^\top c_i - b_{ij} = \frac{1}{2}\|c_i - c_j\|_2, \qquad u_{ij}^\top c_j - b_{ij} = -\frac{1}{2}\|c_i - c_j\|_2.$$

Thus each edge distinction separates the two prototypes that caused it.

5.3 Why graph memory is different from a tree

A tree stores distinctions as a hierarchy. A mistake near the root changes every downstream routing decision. A graph stores distinctions locally between concepts. Creating a new concept does not rewrite old paths; it adds a local alternative and edges explaining why the new concept differs from its neighbors.

A binary distinction tree is recovered by restricting G_t to a rooted binary hierarchy and forcing every input to follow a single path. DGM is therefore the parent data structure; trees are a sparse, hard-routing special case.

5.4 Local memories

The local memory M_j can be chosen according to the task.

- **Classification.** M_j stores label counts or exemplars associated with prototype j.
- **Regression.** M_j stores a local mean, local linear model, or ridge regressor.
- **Associative repair.** M_j stores a Projection Memory matrix W_j and updates it by

$$W_{j,t+1} = W_{j,t} + \frac{\eta}{1 + \eta\|k_t\|_2^2}(v_t - W_{j,t}k_t)k_t^\top.$$

The conceptual rule is unchanged: if the prototype is adequate, repair its local state; if evidence is incompatible with the current prototypes, create a new distinction.

5.5 Language modeling instance

For language modeling, the world state is a prefix and the consequence is the next observed token. Let a frozen or slowly trained backbone produce hidden states

$$h_t = \phi_\theta(x_{\leq t}).$$

DGM is inserted as a runtime memory over hidden states. Each prototype c_j represents a recurring context type: a citation pattern, a local syntax regime, a document-specific entity usage, or a repeated task format. The local memory M_j predicts a correction to the next-token distribution.

A scalable instance stores a target embedding mean

$$r_j = \frac{1}{n_j}\sum_{s \in C_j} E[x_{s+1}],$$

where $E[y] \in \mathbb{R}^{d_e}$ is the output embedding of token y. For a new prefix, DGM retrieves prototypes $N_k(h_t)$ and forms the correction

$$\ell_{\mathrm{DGM}}(y \mid h_t) = \sum_{j \in N_k(h_t)} \alpha_j(h_t) E[y]^\top r_j.$$

The final logits are

$$\ell(y \mid x_{\leq t}) = \ell_\theta(y \mid x_{\leq t}) + \lambda_t \ell_{\mathrm{DGM}}(y \mid h_t).$$

Here λ_t is a learned or fixed trust weight. After the next token x_{t+1} is observed, the machine updates by OBSERVE with consequence $y = x_{t+1}$. If an existing prototype is repaired, its embedding memory can be updated by

$$r_j^+ = \frac{n_j r_j + E[x_{t+1}]}{n_j + 1}, \qquad n_j^+ = n_j + 1.$$

If the observed next token is surprising under the nearest prototype and the context is not well explained by existing prototypes, DGM creates a new prototype at h_t and initializes

$$c_{N_t+1} = h_t, \qquad r_{N_t+1} = E[x_{t+1}], \qquad n_{N_t+1} = 1.$$

This gives a direct test-time learning protocol for language modeling. During streaming prediction, document continuation, user correction, tool feedback, or supervised next-token evaluation, the next observed token is genuine evidence and can refine or repair the runtime memory. During unconstrained generation, the model should not update from its own sampled token by default, because self-generated samples are actions, not external consequences.

Algorithm 3 OBSERVENEXTTOKEN for DGM language modeling

```
Input: prefix x_{≤t}, observed token x_{t+1}, DGM state S_t
h_t ← φ_θ(x_{≤t})
retrieve N_k(h_t) and compute logits ℓ_θ + λ_t ℓ_DGM
j* ← nearest or highest-weight prototype
ρ ← −log p(x_{t+1} | x_{≤t})
if ρ ≤ τ_repair then
    update the local token memory at j*
else
    create a new prototype at h_t with local memory initialized by E[x_{t+1}]
    add distinction edges to confused neighboring prototypes
end if
return updated state
```

5.6 Complexity

With exact nearest-neighbor search, prediction costs

$$O(N_t d + kd + C_{\mathrm{local}}),$$

where C_{local} is the cost of evaluating the selected local memories. With approximate nearest-neighbor indexing, the $N_t d$ term can be replaced by the query cost of the index. Creating a prototype costs $O(d)$ plus the cost of initializing its memory. Adding q distinction edges costs $O(qd)$. If each prototype stores an embedding mean in $\mathbb{R}^{d_e}$, runtime memory is

$$O(N_t(d + d_e) + |E_t|d).$$

Theorem 5.2 (DGM data-structure bound, informal version of Theorem F.2)**.** *For exact search over N_t prototypes in $\mathbb{R}^d$, one DGM prediction costs $O(N_t d + kd + C_{\mathrm{local}})$. One observation that repairs a selected prototype costs this plus the local repair cost. One observation that creates a new prototype and q distinction edges costs an additional $O(d + qd)$. With embedding-mean memories, the runtime state uses $O(N_t(d + d_e) + |E_t|d)$ memory.*

6 DGM Layers for Large-Model Training

The data structure above is useful as a runtime learner, but a large model needs a layer. The layer must satisfy two constraints that pull in opposite directions. Its read computation must be differentiable, so that the backbone can learn to use the memory. Its write computation must be cheap and stable, so that training and inference can use the same update rule without backpropagating through the entire memory history.

6.1 A trainable DGM layer

Let $h_t \in \mathbb{R}^d$ be the hidden state at a Transformer layer. A DGM layer maintains a prototype bank

$$\mathcal{S}_t = \{c_j, A_j, B_j, n_j\}_{j=1}^{N_t},$$

where $c_j \in \mathbb{R}^{d_q}$ is a prototype and $A_j \in \mathbb{R}^{r\times d}$, $B_j \in \mathbb{R}^{d\times r}$ define a rank-r local adapter. The layer first computes a query

$$q_t = Qh_t.$$

It retrieves a sparse neighborhood $N_k(q_t)$ and computes weights

$$\alpha_j(q_t) = \frac{\exp(-\|q_t - c_j\|_2^2/T)}{\sum_{i\in N_k(q_t)} \exp(-\|q_t - c_i\|_2^2/T)}.$$

The read path is

$$\Delta h_t = \sum_{j\in N_k(q_t)} \alpha_j(q_t) B_j A_j h_t, \qquad \tilde{h}_t = h_t + \gamma_\theta(h_t)\Delta h_t.$$

This is a sparse mixture of local low-rank repairs. It is differentiable through the selected memories, routing weights, and backbone parameters. The top-k retrieval itself may be treated as a sparse selection operator, as in mixture-of-experts and retrieval systems.

6.2 Detached write path

After the training target is observed, the layer updates its fast state without placing the entire update history on the gradient tape. For language modeling, the target is x_{t+1}. A simple local write target is the next-token embedding

$$v_t = E[x_{t+1}].$$

Each selected prototype can store a local Projection Memory W_j or update its low-rank adapter using a projected local objective. In the simplest embedding-memory form,

$$r_j^+ = \frac{n_j r_j + \alpha_j(q_t) v_t}{n_j + \alpha_j(q_t)}, \qquad n_j^+ = n_j + \alpha_j(q_t).$$

In the richer adapter form, the local memory is repaired by a rank-one projection update. The update is detached:

$$\mathrm{stopgrad}(\mathcal{S}_{t+1}) = \mathrm{DGMObserve}(\mathrm{stopgrad}(\mathcal{S}_t), h_t, x_{t+1}).$$

Thus the model trains slow parameters through the read path and learns fast state through the same observation rule used at inference.

Algorithm 4 TrainStep for a DGM Transformer layer

Input: token block $x_{1:L}$, slow parameters Θ, DGM state $\mathcal{S}$
for $t = 1, \ldots, L$ **do**
  compute hidden state h_t using the backbone
  $q_t \leftarrow Qh_t$
  retrieve $N_k(q_t)$ from the prototype bank
  read $\Delta h_t \leftarrow \sum_{j \in N_k(q_t)} \alpha_j(q_t) B_j A_j h_t$
  continue the network with $\tilde{h}_t = h_t + \gamma_\theta(h_t)\Delta h_t$
end for
compute next-token loss $\mathcal{L}(\Theta, \mathcal{S})$
update slow parameters Θ by backpropagation through the read path
for $t = 1, \ldots, L$ **do**
  update $\mathcal{S}$ by detached DGMObserve using consequence x_{t+1}
end for
return updated Θ and $\mathcal{S}$

6.3 Why this can scale

The layer is not a replacement for backpropagation through the backbone. It is a replacement for inference-time and history-time backpropagation through the adaptive state. During training, backpropagation sees the selected read computation. During inference, the same layer performs forward retrieval and detached state repair. Therefore the train and inference programs differ only in whether slow parameters receive gradients.

Theorem 6.1 (DGM layer cost, informal version of Theorem F.3)**.** *Consider a DGM layer with hidden dimension d, query dimension d_q, rank r, top-k retrieval, and prototype-bank query cost $C_{\text{ret}}(N_t, d_q)$. For one token, the read overhead is*

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + kdr).$$

If the write path is detached and uses rank-one local repair, the write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + k(d_q + dr)).$$

For a block of length L, the DGM state history need not be stored for backpropagation; the additional activation memory is the selected read computation, not the full sequence of memory states.

The important comparison is with test-time gradient adaptation. A TTT-style layer typically performs an inner backward pass at inference time. A DGM layer performs retrieval, sparse local read, and closed-form or sufficient-statistic repair. The cost is forward-only:

$$\text{DGM inference} = \text{backbone forward} + \text{retrieval} + \text{sparse local repair}.$$

6.4 Distributed training

The prototype bank can be sharded by the same principle used in retrieval and sparse expert systems. Each device owns a subset of prototypes. A query is routed to a small set of shards,

each shard returns local top-k candidates, and a final top-k merge selects the memories used by the read path. Only selected memories receive gradients. Detached writes are sent back to the owning shards.

Budgeting is necessary for long training. A practical DGM layer maintains

$$N_t \leq N_{\max}.$$

When the bank is full, the layer may merge compatible prototypes, evict low-usage prototypes, or compress old prototypes into a shared local memory. These operations are modeling choices rather than new learning principles. The principle remains refine when the representation is too coarse, and repair when the local state is adequate.

6.5 Closed-form adapters

The local memory need not be an exemplar table. A particularly useful large-model adapter is a closed-form ridge readout over the retrieved support memory. Let $Z \in \mathbb{R}^{n\times d}$ be normalized support embeddings stored by a DGM memory and let $Y \in \mathbb{R}^{n\times C}$ be one-hot consequences. The DGM-Ridge adapter computes

$$\Gamma = (ZZ^\top + \lambda I)^{-1}Y, \qquad \mathrm{logits}(z) = zZ^\top\Gamma.$$

This is a projection-style repair: it fits the local adapter in one linear solve rather than by iterated backpropagation. When n is small, as in few-shot adaptation or a retrieved local neighborhood, the solve is cheap. In a large model, the same idea can be applied per prototype, per document, or per retrieved shard.

7 From Distinctions to Projection Memory

DGM uses refinement and repair in the same runtime object. Distinction Learning asks what the learner can tell apart. Projection Learning assumes such a representation and asks how a state inside it should be minimally repaired. This section makes the relation formal in the finite one-hot case.

7.1 One-hot partition memory

Let $\mathcal{P} = \{B_1, \ldots, B_n\}$ and let $\phi_{\mathcal{P}}$ be the one-hot feature map from Section 2. A memory matrix $W \in \mathbb{R}^{m\times n}$ defines a value map

$$f_W(\omega) = W\phi_{\mathcal{P}}(\omega).$$

This is exactly a table over the atoms of $\mathcal{P}$: every state inside the same atom receives the same value.

Theorem 7.1 (One-hot memory is partition-measurable, informal version of Theorem F.4)**.** *For every $W \in \mathbb{R}^{m\times n}$, the map $f_W(\omega) = W\phi_{\mathcal{P}}(\omega)$ is $\mathcal{P}$-measurable. Conversely, every $\mathcal{P}$-measurable map $f : \Omega \to \mathbb{R}^m$ is represented by some W through $f = f_W$.*

Thus a one-hot associative memory is not merely analogous to a partition. It is an exact finite representation of all vector-valued predictors measurable with respect to that partition.

7.2 CPM conflict as missing distinction

Conservative Projection Memory detects conflict when a new key has zero innovation but nonzero residual. In the one-hot finite representation, this diagnostic is exactly the statement that two world states have been placed in the same atom despite requiring different values.

Theorem 7.2 (CPM conflict equals missing distinction, informal version of Theorem F.5)**.** *Let $\omega, \omega' \in B_i$ for an atom $B_i \in \mathcal{P}$, and suppose required values satisfy $v(\omega) \neq v(\omega')$. Let $k = \phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i$. If $We_i = v(\omega)$ has already been committed, then evidence requiring $We_i = v(\omega')$ has CPM innovation $z = 0$ and residual $e \neq 0$.*

The theorem explains why a projection update should stop. The key is not new; the old information structure maps both states to the same coordinate. The residual is nonzero because the consequence differs. The missing update is a refinement of $\mathcal{P}$ that separates ω from ω'.

7.3 The hierarchy

The resulting hierarchy is

$$\text{distinction refinement} \supset \text{projection repair} \supset \text{implicit gradient} \supset \text{explicit gradient descent.}$$

Distinction refinement changes the sigma-algebra $\mathcal{F}_t$. Projection repair changes a state s_t inside a fixed information geometry. Gradient descent is the linearized ℓ_2 special case of soft projection. Backpropagation is the mechanism that computes such gradients in differentiable static graphs.

This is not a rejection of gradient learning or projection memory. It is a separation of roles. When the representation is adequate, repair and optimization are the right tools. When the representation identifies states that consequences require it to separate, the first learning act is to create the missing distinction.

8 Technical Overview

This section gives the proof ideas in the same order as the main text. Full formal versions appear in Appendices D, E, and F.

Theorem 8.1 (Minimal finite refinement, proof sketch for Theorem D.1)**.** *The informal statement is Theorem 3.1.*

Proof sketch. Every atom of $\mathcal{P}_E$ lies inside an atom of $\mathcal{P}$, so $\mathcal{P}_E$ refines $\mathcal{P}$. Also

$$E = \bigcup_{B \in \mathcal{P}} (B \cap E),$$

so E is $\mathcal{P}_E$-measurable. If $\mathcal{Q}$ refines $\mathcal{P}$ and makes E measurable, then every atom $Q \in \mathcal{Q}$ satisfies

$$Q \subseteq B$$

for some $B \in \mathcal{P}$, and also either $Q \subseteq E$ or $Q \subseteq E^c$. Hence

$$Q \subseteq B \cap E \qquad \text{or} \qquad Q \subseteq B \cap E^c.$$

Thus $\mathcal{Q}$ refines $\mathcal{P}_E$. □

Theorem 8.2 (Minimal sigma-algebra refinement, proof sketch for Theorem D.2)**.** *The informal statement is Theorem 3.2.*

Proof sketch. By definition,

$$\mathcal{F} \vee \sigma(Z) = \sigma(\mathcal{F} \cup \sigma(Z)).$$

This sigma-algebra contains $\mathcal{F}$ and makes Z measurable. If $\mathcal{G}$ also contains $\mathcal{F}$ and makes Z measurable, then

$$\mathcal{F} \cup \sigma(Z) \subseteq \mathcal{G}.$$

Since $\mathcal{G}$ is a sigma-algebra,

$$\sigma(\mathcal{F} \cup \sigma(Z)) \subseteq \mathcal{G}.$$

□

Theorem 8.3 (Contradiction, proof sketch for Theorem D.3)**.** *The informal statement is Theorem 3.3.*

Proof sketch. If f is $\mathcal{P}$-measurable and ω, ω' lie in the same atom, then

$$f(\omega) = f(\omega').$$

If $f(\omega) = y(\omega)$ and $f(\omega') = y(\omega')$, then

$$y(\omega) = y(\omega'),$$

contradicting the hypothesis. □

Theorem 8.4 (Risk monotonicity, proof sketch for Theorem E.1)**.** *The informal statement is Theorem 4.1.*

Proof sketch. If $\mathcal{F} \subseteq \mathcal{G}$, then every $\mathcal{F}$-measurable action is $\mathcal{G}$-measurable. Therefore

$$\{a : a \text{ is } \mathcal{F}\text{-measurable}\} \subseteq \{a : a \text{ is } \mathcal{G}\text{-measurable}\}.$$

Taking the infimum of the same expected loss over the larger set gives

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

□

Theorem 8.5 (Log-loss value, proof sketch for Theorem E.2)**.** *The informal statement is Theorem 4.2.*

Proof sketch. For any $\mathcal{F}$-measurable conditional distribution q,

$$\mathbb{E}[-\log q(Y)] = H(Y \mid \mathcal{F}) + \mathbb{E}\left[D_{\mathrm{KL}}(\mathbb{P}(Y \in \cdot \mid \mathcal{F}) \| q(\cdot))\right].$$

The minimizer is $q^\star = \mathbb{P}(Y \in \cdot \mid \mathcal{F})$, so $R_{\log}(\mathcal{F}) = H(Y \mid \mathcal{F})$. Similarly $R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{G})$. Therefore

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}).$$

□

Theorem 8.6 (Squared-loss value, proof sketch for Theorem E.3)**.** *The informal statement is Theorem 4.3.*

Proof sketch. The optimal $\mathcal{F}$-predictor is $\mathbb{E}[Y \mid \mathcal{F}]$. Since $\mathcal{F} \subseteq \mathcal{G}$,

$$Y - \mathbb{E}[Y \mid \mathcal{F}] = (Y - \mathbb{E}[Y \mid \mathcal{G}]) + (\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]) .$$

The two terms are orthogonal because the first has conditional mean zero given $\mathcal{G}$ and the second is $\mathcal{G}$-measurable. Taking squared ℓ_2 norms and expectations gives the identity. □

Theorem 8.7 (Prototype edge separation, proof sketch for Theorem F.1)**.** *The informal statement is Theorem 5.1.*

Proof sketch. With $u = (c_i - c_j)/\left\|c_i - c_j\right\|_2$ and $b = u^\top(c_i + c_j)/2$,

$$\begin{aligned} u^\top c_i - b &= \frac{1}{2}u^\top(c_i - c_j) = \frac{1}{2}\left\|c_i - c_j\right\|_2 , \\ u^\top c_j - b &= -\frac{1}{2}u^\top(c_i - c_j) = -\frac{1}{2}\left\|c_i - c_j\right\|_2 . \end{aligned}$$

□

Theorem 8.8 (DGM data structure, proof sketch for Theorem F.2)**.** *The informal statement is Theorem 5.2.*

Proof sketch. A prediction computes distances from h to N_t prototypes in $\mathbb{R}^d$, costing

$$O(N_t d).$$

It then evaluates k selected local memories, costing $O(kd + C_{\text{local}})$ including sparse routing weights. Repair adds the local repair cost. Creating one prototype stores a vector in $\mathbb{R}^d$. Adding q distinction edges stores q vectors and thresholds, costing

$$O(qd).$$

For embedding-mean memories, each prototype stores $c_j \in \mathbb{R}^d$ and $r_j \in \mathbb{R}^{d_e}$, while each edge stores one distinction vector in $\mathbb{R}^d$. Hence memory is

$$O(N_t(d + d_e) + |E_t|d).$$

□

Theorem 8.9 (DGM layer cost, proof sketch for Theorem F.3)**.** *The informal statement is Theorem 6.1.*

Proof sketch. Computing $q = Qh$ costs $O(dd_q)$. Retrieval costs $C_{\text{ret}}(N_t, d_q)$ by definition. For each selected prototype, a rank-r adapter computes

$$B_j A_j h.$$

The two matrix-vector products cost $O(dr)$, so k selected adapters cost

$$O(kdr).$$

The detached write path repeats query and retrieval, then updates only selected local states. A rank-one local repair or low-rank sufficient-statistic update costs $O(dr)$ per selected memory, with $O(d_q)$ additional prototype bookkeeping. Hence write overhead is

$$O(dd_q + C_{\mathrm{ret}}(N_t, d_q) + k(d_q + dr)).$$

Because the write path is detached, backpropagation does not need intermediate copies of $\mathcal{S}_t$ for all t. It stores the selected read computation used in the forward loss. □

Theorem 8.10 (One-hot memory, proof sketch for Theorem F.4)**.** *The informal statement is Theorem 7.1.*

Proof sketch. If $\omega, \omega' \in B_i$, then both one-hot features equal e_i, so

$$W\phi_{\mathcal{P}}(\omega) = We_i = W\phi_{\mathcal{P}}(\omega').$$

Thus $W\phi_{\mathcal{P}}$ is constant on atoms. Conversely, if f is constant on each B_i, put that constant value in column i of W. Then

$$f(\omega) = W\phi_{\mathcal{P}}(\omega)$$

for every ω. □

Theorem 8.11 (CPM conflict, proof sketch for Theorem F.5)**.** *The informal statement is Theorem 7.2.*

Proof sketch. If $\omega, \omega' \in B_i$, then

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

After committing $We_i = v(\omega)$, the later evidence $We_i = v(\omega')$ has the same key. Its innovation relative to the old key span is

$$z = (I - e_i e_i^\top)e_i = 0.$$

Its residual is

$$e = v(\omega') - We_i = v(\omega') - v(\omega) \neq 0.$$

□

9 Computational Checks

The theory above is structural, but the model is meant to be executable. We include small reproducible comparisons against backpropagation. The accompanying script is in the experiment directory and uses PyTorch for the backpropagation baselines and NumPy for the distinction-memory methods.

The first task is a two-context XOR stream. In the first context the label is XOR of two Gaussian cluster signs; in the second context the labels are flipped. The context coordinate is observable, but the learner must discover the relevant distinction. We compare three learners. The offline MLP is trained by ordinary backpropagation on the full stream for 500 epochs and should be read as a batch upper bound, not as a runtime learner. The online MLP predicts each example

Model	Runtime update	Prequential accuracy	Test accuracy
Backprop MLP, offline	batch backprop	–	0.993 ± 0.002
Backprop MLP, online	one backprop step	0.569 ± 0.065	0.654 ± 0.038
DGM-style memory, online	refine-or-repair	0.804 ± 0.025	0.944 ± 0.026

Table 1: DGM-style distinction memory versus backpropagation on a limited online contextual-XOR stream. Offline backprop is a batch upper bound. The relevant runtime comparison is between online backpropagation and online distinction refinement. The distinction-memory run has monotone objective decrease at accepted refinements in all runs.

Dataset	Offline backprop	Online backprop	Online DGM-style memory
MNIST	0.843 ± 0.009	0.682 ± 0.038	0.851 ± 0.003
CIFAR-10	0.273 ± 0.003	0.137 ± 0.010	0.243 ± 0.008

Table 2: Image-stream checks. The point is not to beat fully trained image models, but to compare one-pass runtime updates. Prototype/exemplar distinction memory beats one-step online backpropagation in both streams and is competitive with the small offline MLP on MNIST.

and then performs one backpropagation step on that example. The DGM-style distinction memory uses the same online protocol: predict first, then observe and refine-or-repair. Results are averaged over five seeds with 100 examples per context.

We also run small online image-stream checks on MNIST and CIFAR-10 using cached local datasets. In these experiments the distinction-memory method uses exemplar/prototype repair. The online backpropagation baseline sees each example once and performs one gradient step. Offline backpropagation is again a batch upper bound.

These experiments test a specific thesis rather than a broad benchmark claim. When the representation lacks a useful distinction, one gradient step on the current example is a weak runtime update. A distinction-memory system can instead create or retrieve a local concept and then repair local state. With full offline access and many epochs, backpropagation remains strong, as expected.

For a stronger adapter benchmark, we train a shared convolutional encoder and freeze it. The adaptation problem is then deliberately narrow: given only five labeled support examples per class, adapt the readout. We compare DGM-Ridge, a closed-form projection adapter from Section 6, against a one-pass backpropagation head and a 200-step backpropagation head on the same frozen embeddings. This isolates the large-model use case: a strong backbone already exists, and the question is whether runtime adaptation should be a gradient loop or a closed-form memory repair.

Finally, we include a synthetic language-modeling stream. Each document context induces a different next-token transition map. The learner observes a prefix representation and must predict the next token before seeing it. After the next token is revealed, DGM-style memory stores the prefix type and next-token consequence; online backpropagation again receives one gradient step per token.

We also verify the projection-memory residual contraction identity used by local repair. For random W, k, v and step size η, the soft Projection Memory update satisfies

$$v - W_{t+1}k = \frac{1}{1 + \eta \|k\|_2^2}(v - W_t k).$$

Over 200 random trials, the maximum absolute error between the measured residual ratio and the

Dataset	Method	Accuracy	Adapter time
MNIST, 5-shot	one-pass backprop head	0.100 ± 0.000	0.0002s
MNIST, 5-shot	200-step backprop head	0.332 ± 0.009	0.0320s
MNIST, 5-shot	DGM-Ridge	$\mathbf{0.623 \pm 0.003}$	**0.0018**s
CIFAR-10, 5-shot	one-pass backprop head	0.115 ± 0.014	0.0003s
CIFAR-10, 5-shot	200-step backprop head	0.359 ± 0.009	0.0325s
CIFAR-10, 5-shot	DGM-Ridge	$\mathbf{0.397 \pm 0.010}$	**0.0004**s

Table 3: Few-shot adapter benchmark on frozen embeddings. DGM-Ridge fits the adapter by a closed-form projection solve. It is substantially faster than the tuned backpropagation head and achieves higher accuracy in this local run. The one-pass head is slightly cheaper but does not learn a useful classifier.

Task	Offline backprop	Online backprop	Online DGM-style memory
Synthetic LM	0.931 ± 0.071	0.125 ± 0.021	0.912 ± 0.077

Table 4: Synthetic next-token stream. The task is designed to isolate runtime adaptation: the hidden context determines the token transition rule, so the useful operation is to remember a local prefix-to-token consequence after observing it. DGM-style memory approaches the offline batch MLP while one-step online backpropagation adapts slowly.

formula was $3.33 \cdot 10^{-16}$.

10 Conclusion

Distinction Graph Machines turn the distinction principle into an explicit runtime learner. The model has prototypes, local memories, distinction edges, repair rules, and a train/test API. Its guiding rule is simple: refine when the concepts are wrong, repair when the local state is wrong.

The theory explains why this separation matters. Contradictions inside an atom cannot be solved by optimizing a measurable predictor. They require refinement. Once the local concept is adequate, its state can be repaired by sufficient-statistic, exemplar, ridge, or projection-memory updates. DGM turns that separation into a concrete runtime state: prototypes hold local memories, and edges record distinctions forced by conflict.

The resulting picture places gradient learning, projection repair, and distinction refinement in one hierarchy. Backpropagation moves slow parameters; projection memory repairs fast state; DGM adds the missing operation of changing the information structure on which those states are defined. In language modeling, this means that a runtime learner need not backpropagate through the entire model at inference time: it can maintain hidden-state prototypes, repair next-token memories, and create new distinctions when the stream reveals a genuinely new context.

References

[AW01] Katy S. Azoury and Manfred K. Warmuth. Relative loss bounds for on-line density estimation with the exponential family of distributions. *Machine Learning*, 43:211–246, 2001.

[BHM+16] Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

[Bre67] Lev M. Bregman. The relaxation method of finding the common point of convex sets and its application to the solution of problems in convex programming. *USSR Computational Mathematics and Mathematical Physics*, 7(3):200–217, 1967.

[CDK+06] Koby Crammer, Ofer Dekel, Joseph Keshet, Shai Shalev-Shwartz, and Yoram Singer. Online passive-aggressive algorithms. *Journal of Machine Learning Research*, 7:551–585, 2006.

[Haz16] Elad Hazan. *Introduction to Online Convex Optimization*. Foundations and Trends in Optimization, 2016.

[JCO+17] Max Jaderberg, Wojciech Marian Czarnecki, Simon Osindero, Oriol Vinyals, Alex Graves, David Silver, and Koray Kavukcuoglu. Decoupled neural interfaces using synthetic gradients. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1627–1635, 2017.

[KVPF20] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165, 2020.

[RHW86] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.

[SIS21] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9355–9366, 2021.

[SLD+25] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): RNNs with expressive hidden states. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 57503–57522, 2025.

[SS12] Shai Shalev-Shwartz. Online learning and online convex optimization. *Foundations and Trends in Machine Learning*, 4(2):107–194, 2012.

[SWL+20] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9229–9248, 2020.

[Vov97] Volodya Vovk. Competitive on-line linear regression. In *Advances in Neural Information Processing Systems*, volume 10, 1997.

[Zin03] Martin Zinkevich. Online convex programming and generalized infinitesimal gradient ascent. In *Proceedings of the 20th International Conference on Machine Learning*, pages 928–936, 2003.

Appendix

A Appendix Roadmap

The appendix follows the same story as the main text. It first states the limitations of the refinement view, then situates the paper relative to prior work, and then gives formal statements and proofs.

- **Part 1.** Appendix B clarifies what the theory proves and what it leaves to modeling, statistics, and experiments.
- **Part 2.** Appendix C places the paper relative to prototype methods, decision trees, information theory, sigma-algebra refinement, backpropagation, projection learning, and test-time adaptation.
- **Part 3.** Appendix D proves the minimal refinement and contradiction theorems.
- **Part 4.** Appendix E proves monotonicity of Bayes risk and the exact log-loss and squared-loss refinement identities.
- **Part 5.** Appendix F proves the algorithmic and bridge guarantees: prototype edge separation, DGM data-structure complexity, DGM layer cost, and the bridge between partition refinement and CPM conflict.

B Limitations

The theory identifies when a distinction is necessary or valuable at the population-information level. It does not solve the search problem of finding the best distinction in a rich hypothesis class. The online algorithm is greedy and local; its refinement rule concerns local prototypes and their nearby conflicts, not global optimality over all information structures.

The risk identities assume correctly estimated conditional distributions or conditional means. With finite data, refinement can increase estimation error, so useful population distinctions may be statistically unsafe. Complexity penalties, thresholds, validation, Bayesian priors, or minimum leaf sizes are needed in practical systems.

The DGM method specifies one canonical refinement mechanism, based on prototypes and local distinction edges, but this is not claimed to be optimal. Rich domains may require learned prototype creation rules, approximate nearest-neighbor indices, nonlinear distinction edges, merge operations, forgetting, or budgeted memory management. The bridge to Projection Learning and CPM is exact for one-hot finite partition memories. Continuous neural representations need additional modeling choices: when a continuous conflict should create a new prototype, how local projection memories should be inherited, and how refined information structures interact with differentiable backbones.

The large-model layer should be read as an architectural path, not as a completed scaling theorem. It gives a trainable read path and a detached write path with explicit complexity, but it does not prove optimization guarantees for deep nonconvex backbones, nor does it solve distributed systems issues such as load balancing, stale prototype shards, or memory-bank budgeting.

C Related Work

Prototype and memory methods. Prototype and memory methods represent data by stored exemplars or centers. DGM uses the same computational intuition but adds a refinement rule: a

new prototype is created when local repair cannot absorb evidence without conflating consequences, and distinction edges record the separating evidence.

Decision trees and splitting criteria. Classical decision trees use information gain and variance reduction as splitting criteria. In our view, a decision tree is a hard-routing special case of a distinction graph. It stores distinctions in a hierarchy rather than as local edges between concepts.

Sparse experts and retrieval layers. Mixture-of-experts and retrieval-augmented models scale neural computation by selecting a small subset of parameters or memories for each input. A DGM layer has a similar sparse read path, but the selected objects are runtime concepts with local memories, and the write path can refine or repair the state after observing consequences.

Information theory and statistical decision theory. The log-loss identity is the standard relation between optimal code length and conditional entropy, while the squared-loss identity is the orthogonal decomposition of conditional expectations in L^2. Our contribution is to place these identities inside a runtime-learning story: the value of learning a distinction is its reduction in Bayes risk.

Backpropagation and gradient learning. Backpropagation supplies differentiable credit assignment for fixed computation graphs [RHW86]. Distinction Learning addresses a different failure mode: the current information structure may not contain the distinctions needed for any measurable predictor to succeed.

Projection learning and online repair. Bregman projection, mirror descent, and proximal methods provide minimal-change state updates [Bre67, Zin03, SS12, Haz16]. Conservative Projection Memory uses this idea for runtime associative repair. The present paper demotes projection repair to the continuous-state layer that operates after the relevant information structure has been fixed.

Test-time adaptation. Test-time training adapts parameters using auxiliary losses at inference time [SWL+20]. TTT layers treat hidden states as trainable models updated on test sequences [SLD+25]. Distinction Learning suggests a complementary diagnostic: when runtime evidence conflicts inside an indistinguishable atom, the issue is not only how to update fast state, but what distinction the state failed to represent.

D Minimal Refinement Proofs

Theorem D.1 (Minimal finite refinement, formal version of Theorem 3.1)**.** *Let Ω be finite, let $\mathcal{P}$ be a partition of Ω, and let $E \subseteq \Omega$. Define*

$$\mathcal{P}_E = \{B \cap E : B \in \mathcal{P}, B \cap E \neq \varnothing\} \cup \{B \cap E^c : B \in \mathcal{P}, B \cap E^c \neq \varnothing\}.$$

Then:

- **Part 1.** *$\mathcal{P}_E$ is a partition of Ω.*
- **Part 2.** *$\mathcal{P}_E$ refines $\mathcal{P}$.*

- **Part 3.** *E is $\mathcal{P}_E$-measurable.*
- **Part 4.** *If $\mathcal{Q}$ refines $\mathcal{P}$ and E is $\mathcal{Q}$-measurable, then $\mathcal{Q}$ refines $\mathcal{P}_E$.*

Consequently, $\mathcal{P}_E$ is the unique minimal refinement of $\mathcal{P}$ making E measurable.

Proof. For each $B \in \mathcal{P}$, the nonempty sets among $B \cap E$ and $B \cap E^c$ are disjoint and have union B. Since the atoms of $\mathcal{P}$ are disjoint and cover Ω, the atoms of $\mathcal{P}_E$ are disjoint and cover Ω. Thus Part 1 holds.

Every atom of $\mathcal{P}_E$ is contained in some $B \in \mathcal{P}$, so Part 2 holds. Also

$$E = \bigcup_{B \in \mathcal{P}} (B \cap E),$$

where empty intersections are ignored. Hence E is a union of atoms of $\mathcal{P}_E$, proving Part 3.

Let $\mathcal{Q}$ refine $\mathcal{P}$ and make E measurable. For any atom $Q \in \mathcal{Q}$, there exists $B \in \mathcal{P}$ with

$$Q \subseteq B.$$

Since E is $\mathcal{Q}$-measurable, each atom of $\mathcal{Q}$ is either contained in E or disjoint from E. Hence

$$Q \subseteq B \cap E \qquad \text{or} \qquad Q \subseteq B \cap E^c.$$

Thus each atom of $\mathcal{Q}$ is contained in an atom of $\mathcal{P}_E$, so $\mathcal{Q}$ refines $\mathcal{P}_E$. This proves Part 4.

If another partition has the same minimal property, it refines $\mathcal{P}_E$ and is refined by $\mathcal{P}_E$, so the two partitions have the same atoms. □

Theorem D.2 (Minimal sigma-algebra refinement, formal version of Theorem 3.2)**.** *Let $(\Omega, \mathcal{A})$ be a measurable space, let $\mathcal{F} \subseteq \mathcal{A}$ be a sigma-algebra, and let $Z : \Omega \to \mathcal{Z}$ be a measurable map into a measurable space $(\mathcal{Z}, \mathcal{B})$. The join $\mathcal{F} \vee \sigma(Z)$ is the smallest sigma-algebra $\mathcal{G} \subseteq \mathcal{A}$ such that $\mathcal{F} \subseteq \mathcal{G}$ and Z is $\mathcal{G}$-measurable.*

Proof. By definition,

$$\mathcal{F} \vee \sigma(Z) = \sigma(\mathcal{F} \cup \sigma(Z)).$$

Therefore $\mathcal{F} \subseteq \mathcal{F} \vee \sigma(Z)$ and $\sigma(Z) \subseteq \mathcal{F} \vee \sigma(Z)$, so Z is measurable with respect to $\mathcal{F} \vee \sigma(Z)$.

Let $\mathcal{G}$ be any sigma-algebra such that $\mathcal{F} \subseteq \mathcal{G}$ and Z is $\mathcal{G}$-measurable. The latter condition means $\sigma(Z) \subseteq \mathcal{G}$. Hence

$$\mathcal{F} \cup \sigma(Z) \subseteq \mathcal{G}.$$

Since $\mathcal{G}$ is a sigma-algebra,

$$\sigma(\mathcal{F} \cup \sigma(Z)) \subseteq \mathcal{G}.$$

Thus $\mathcal{F} \vee \sigma(Z) \subseteq \mathcal{G}$. □

Theorem D.3 (Contradiction requires distinction, formal version of Theorem 3.3)**.** *Let $\mathcal{P}$ be a partition of Ω and let $y : \Omega \to A$ be a required output. Suppose there exist $\omega, \omega' \in B$ for some atom $B \in \mathcal{P}$ such that $y(\omega) \neq y(\omega')$. Then no $\mathcal{P}$-measurable $f : \Omega \to A$ satisfies both*

$$f(\omega) = y(\omega), \qquad f(\omega') = y(\omega').$$

Proof. Since f is $\mathcal{P}$-measurable and $\omega, \omega' \in B$,

$$f(\omega) = f(\omega').$$

If both requirements held, then

$$y(\omega) = f(\omega) = f(\omega') = y(\omega'),$$

contradicting $y(\omega) \neq y(\omega')$. □

E Value of Distinction Proofs

Theorem E.1 (Bayes-risk monotonicity, formal version of Theorem 4.1)**.** *Let Y be a target random variable, let A be an action space, and let L be an integrable loss. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_L(\mathcal{G}) \leq R_L(\mathcal{F}).$$

Proof. Define

$$\mathcal{D}_{\mathcal{F}} = \{a : \Omega \to A : a \text{ is } \mathcal{F}\text{-measurable}\}, \qquad \mathcal{D}_{\mathcal{G}} = \{a : \Omega \to A : a \text{ is } \mathcal{G}\text{-measurable}\}.$$

Since $\mathcal{F} \subseteq \mathcal{G}$,

$$\mathcal{D}_{\mathcal{F}} \subseteq \mathcal{D}_{\mathcal{G}}.$$

Therefore

$$\begin{aligned} R_L(\mathcal{G}) &= \inf_{a \in \mathcal{D}_{\mathcal{G}}} \mathbb{E}[L(Y, a)] \\ &\leq \inf_{a \in \mathcal{D}_{\mathcal{F}}} \mathbb{E}[L(Y, a)] \\ &= R_L(\mathcal{F}). \end{aligned}$$

□

Theorem E.2 (Log-loss value, formal version of Theorem 4.2)**.** *Let Y be finite and let $\mathcal{F} \subseteq \mathcal{G}$. Under log loss, an admissible action is a measurable conditional distribution $q(\cdot)$ on the range of Y, and the loss is $-\log q(Y)$. Then*

$$R_{\log}(\mathcal{F}) - R_{\log}(\mathcal{G}) = I(Y; \mathcal{G} \mid \mathcal{F}).$$

Proof. For any $\mathcal{F}$-measurable conditional distribution q, write $p_{\mathcal{F}}(y) = \mathbb{P}(Y = y \mid \mathcal{F})$. Then

$$\begin{aligned} \mathbb{E}[-\log q(Y) \mid \mathcal{F}] &= \sum_y p_{\mathcal{F}}(y)(-\log q(y)) \\ &= \sum_y p_{\mathcal{F}}(y)(-\log p_{\mathcal{F}}(y)) + \sum_y p_{\mathcal{F}}(y) \log \frac{p_{\mathcal{F}}(y)}{q(y)} \\ &= H(Y \mid \mathcal{F}) + D_{\mathrm{KL}}(p_{\mathcal{F}} \| q). \end{aligned}$$

Taking expectation and minimizing over q gives

$$R_{\log}(\mathcal{F}) = H(Y \mid \mathcal{F}).$$

Likewise,

$$R_{\log}(\mathcal{G}) = H(Y \mid \mathcal{G}).$$

By definition of conditional mutual information between Y and the additional information $\mathcal{G}$ given $\mathcal{F}$,

$$I(Y; \mathcal{G} \mid \mathcal{F}) = H(Y \mid \mathcal{F}) - H(Y \mid \mathcal{G}).$$

Combining proves the claim. □

Theorem E.3 (Squared-loss value, formal version of Theorem 4.3)**.** *Let $Y \in L^2(\Omega; \mathbb{R}^d)$ and let $L(Y, a) = \|Y - a\|_2^2$. If $\mathcal{F} \subseteq \mathcal{G}$, then*

$$R_2(\mathcal{F}) - R_2(\mathcal{G}) = \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].$$

Proof. The optimal $\mathcal{F}$-measurable predictor is $\mathbb{E}[Y \mid \mathcal{F}]$, and the optimal $\mathcal{G}$-measurable predictor is $\mathbb{E}[Y \mid \mathcal{G}]$. Let

$$U = Y - \mathbb{E}[Y \mid \mathcal{G}], \qquad V = \mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}].$$

Then

$$Y - \mathbb{E}[Y \mid \mathcal{F}] = U + V.$$

Since V is $\mathcal{G}$-measurable and $\mathbb{E}[U \mid \mathcal{G}] = 0$,

$$\mathbb{E}[\langle U, V\rangle \mid \mathcal{G}] = \langle \mathbb{E}[U \mid \mathcal{G}], V\rangle = 0.$$

Thus

$$\begin{aligned}
\mathbb{E}\,\|Y - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2 &= \mathbb{E}\,\|U + V\|_2^2 \\
&= \mathbb{E}\,\|U\|_2^2 + \mathbb{E}\,\|V\|_2^2 \\
&= R_2(\mathcal{G}) + \mathbb{E}\left[\|\mathbb{E}[Y \mid \mathcal{G}] - \mathbb{E}[Y \mid \mathcal{F}]\|_2^2\right].
\end{aligned}$$

Rearranging proves the identity. □

Proposition E.4 (Finite squared-loss split identity)**.** *Let $B = B_0 \cup B_1$ be a disjoint split of a finite sample set. With empirical means $\hat{\mu}_B, \hat{\mu}_{B_0}, \hat{\mu}_{B_1}$ and* SSE *as in Section 4,*

$$\mathrm{SSE}(B) - \mathrm{SSE}(B_0) - \mathrm{SSE}(B_1) = |B_0|\,\|\hat{\mu}_{B_0} - \hat{\mu}_B\|_2^2 + |B_1|\,\|\hat{\mu}_{B_1} - \hat{\mu}_B\|_2^2.$$

Proof. For each child B_b,

$$\begin{aligned}
\sum_{j \in B_b} \|y_j - \hat{\mu}_B\|_2^2 &= \sum_{j \in B_b} \|y_j - \hat{\mu}_{B_b} + \hat{\mu}_{B_b} - \hat{\mu}_B\|_2^2 \\
&= \sum_{j \in B_b} \|y_j - \hat{\mu}_{B_b}\|_2^2 + |B_b|\,\|\hat{\mu}_{B_b} - \hat{\mu}_B\|_2^2,
\end{aligned}$$

because

$$\sum_{j \in B_b} (y_j - \hat{\mu}_{B_b}) = 0.$$

Summing over $b \in \{0, 1\}$ gives the result. □

F DGM Algorithm and Projection-Bridge Proofs

Theorem F.1 (Prototype edge separation, formal version of Theorem 5.1)**.** *Let $c_i, c_j \in \mathbb{R}^d$ with $c_i \neq c_j$. Define*

$$u = \frac{c_i - c_j}{\|c_i - c_j\|_2}, \qquad b = \frac{1}{2} u^\top (c_i + c_j).$$

Then

$$u^\top c_i - b = \frac{1}{2} \|c_i - c_j\|_2, \qquad u^\top c_j - b = -\frac{1}{2} \|c_i - c_j\|_2.$$

Proof. Compute

$$\begin{aligned} u^\top c_i - b &= u^\top c_i - \frac{1}{2} u^\top c_i - \frac{1}{2} u^\top c_j \\ &= \frac{1}{2} u^\top (c_i - c_j) \\ &= \frac{1}{2} \|c_i - c_j\|_2. \end{aligned}$$

Similarly,

$$\begin{aligned} u^\top c_j - b &= u^\top c_j - \frac{1}{2} u^\top c_i - \frac{1}{2} u^\top c_j \\ &= -\frac{1}{2} u^\top (c_i - c_j) \\ &= -\frac{1}{2} \|c_i - c_j\|_2. \end{aligned}$$

□

Theorem F.2 (DGM data-structure bound, formal version of Theorem 5.2)**.** *Assume DGM has N_t prototypes in $\mathbb{R}^d$, exact nearest-prototype search, sparse neighborhood size k, and local-memory evaluation cost C_{local}. Then one prediction costs $O(N_t d + kd + C_{\text{local}})$. One observation that repairs a selected prototype costs this plus the local repair cost. One observation that creates a new prototype and q distinction edges costs an additional $O(d + qd)$. If each prototype stores an embedding mean in $\mathbb{R}^{d_e}$ and each edge stores a linear distinction in $\mathbb{R}^d$, then the runtime state uses $O(N_t(d + d_e) + |E_t| d)$ memory.*

Proof. A distance computation between $h \in \mathbb{R}^d$ and one prototype costs $O(d)$. Exact search over N_t prototypes therefore costs

$$O(N_t d).$$

After the k nearest prototypes are selected, computing sparse routing weights and evaluating selected local memories costs

$$O(kd + C_{\text{local}}).$$

Thus prediction costs

$$O(N_t d + kd + C_{\text{local}}).$$

Observation begins with the same retrieval computation. If it repairs a selected prototype, it adds the local repair cost. If it creates a new prototype, storing the new center costs $O(d)$. For each new edge, computing and storing the linear distinction vector costs $O(d)$, so q edges cost

$$O(qd).$$

The total additional cost for refinement is therefore $O(d + qd)$.

For memory, each prototype stores $c_j \in \mathbb{R}^d$ and $r_j \in \mathbb{R}^{d_e}$, giving

$$O(N_t(d + d_e)).$$

Each edge stores one distinction vector and threshold, giving

$$O(|E_t|d).$$

Adding the two terms proves the claimed bound. □

Theorem F.3 (DGM layer cost, formal version of Theorem 6.1)**.** *Consider a DGM layer with hidden dimension d, query dimension d_q, rank r, top-k retrieval, and prototype-bank query cost $C_{\text{ret}}(N_t, d_q)$. For one token, the read overhead is*

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + kdr).$$

If the write path is detached and uses rank-one local repair, the write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + k(d_q + dr)).$$

For a block of length L, backpropagation through the slow parameters does not require storing the full DGM state history $\mathcal{S}_1, \ldots, \mathcal{S}_L$.

Proof. For a token hidden state $h \in \mathbb{R}^d$, the query computation

$$q = Qh$$

costs $O(dd_q)$. The retrieval cost is $C_{\text{ret}}(N_t, d_q)$ by definition.

For a selected prototype j, the rank-r adapter read is

$$B_j A_j h.$$

The product $A_j h$ costs $O(rd)$, and the product $B_j(A_j h)$ costs $O(dr)$. Therefore one selected adapter costs $O(dr)$ and k selected adapters cost

$$O(kdr).$$

The routing weights add lower-order top-k normalization cost, absorbed into the same bound. This proves the read overhead.

The write path first computes the same query and retrieval. It then updates selected prototype statistics and selected local memories. Prototype bookkeeping costs $O(d_q)$ per selected prototype. A rank-one local repair of a rank-r or low-rank sufficient-statistic memory costs $O(dr)$ per selected memory. Hence the write overhead is

$$O(dd_q + C_{\text{ret}}(N_t, d_q) + k(d_q + dr)).$$

Finally, the update rule is applied as

$$\text{stopgrad}(\mathcal{S}_{t+1}) = \text{DGMObserve}(\text{stopgrad}(\mathcal{S}_t), h_t, x_{t+1}).$$

Thus the computational graph for the loss contains the selected read computation, but not the chain of state-update maps from $\mathcal{S}_1$ through $\mathcal{S}_L$. Therefore full DGM state history is not required for backpropagation through the slow parameters. □

Theorem F.4 (One-hot memory is partition-measurable, formal version of Theorem 7.1)**.** *Let $\mathcal{P} = \{B_1, \ldots, B_n\}$ be a finite partition and let $\phi_{\mathcal{P}}(\omega) = e_i$ for $\omega \in B_i$. For every $W \in \mathbb{R}^{m\times n}$, the map $f_W(\omega) = W\phi_{\mathcal{P}}(\omega)$ is $\mathcal{P}$-measurable. Conversely, every $\mathcal{P}$-measurable map $f : \Omega \to \mathbb{R}^m$ is equal to f_W for some $W \in \mathbb{R}^{m\times n}$.*

Proof. If $\omega, \omega' \in B_i$, then

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

Thus

$$f_W(\omega) = We_i = f_W(\omega'),$$

so f_W is constant on each atom and is $\mathcal{P}$-measurable.

Conversely, if f is $\mathcal{P}$-measurable, it is constant on each B_i. Let w_i be this constant value on B_i, and define

$$W = [w_1, \ldots, w_n].$$

Then for $\omega \in B_i$,

$$W\phi_{\mathcal{P}}(\omega) = We_i = w_i = f(\omega).$$

□

Theorem F.5 (CPM conflict equals missing distinction, formal version of Theorem 7.2)**.** *Let $\mathcal{P} = \{B_1, \ldots, B_n\}$ and let $\omega, \omega' \in B_i$. Suppose $v(\omega) \neq v(\omega')$. Let $k = \phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i$. If a memory W has already committed $We_i = v(\omega)$, then the evidence $We_i = v(\omega')$ has innovation $z = 0$ relative to the old key span and residual $e \neq 0$.*

Proof. Since $\omega, \omega' \in B_i$,

$$\phi_{\mathcal{P}}(\omega) = \phi_{\mathcal{P}}(\omega') = e_i.$$

The old key span contains e_i. Therefore the innovation of the new key $k = e_i$ relative to the old key matrix $K = e_i$ is

$$z = (I - KK^{\dagger})k = (I - e_i e_i^{\top})e_i = 0.$$

The new residual is

$$e = v(\omega') - We_i = v(\omega') - v(\omega).$$

Since $v(\omega') \neq v(\omega)$, we have $e \neq 0$. □